Week 5 Assignment

Data Structure

Chris Brilianto Chandra
Informatika *(Kampus Kota Makassar)*
Universitas Ciputra Surabaya (Kampus Kota Makassar)
cbrilianto@student.ciputra.ac.id

I. Question Number 1 : Bracket Checker

- **Problem Summary**

Disini kami diperlukan untuk membuat sebuah program yang akan menvalidasi tanda kurung yang ada dalam string, entah itu kurung biasa, kurung array ataupun kurung kurawal. Yang perlu diselesaikan disini itu caranya untuk mengimplementasikan proses itu pakai Stack dan memastikan urutan penutupannya sesuai

- **Stack Design**

Di stack ada 2 function utama yang digunakan, yakni Push & Pop. Awalnya push digunakan untuk menaruh data yang adfa dari string masuk kedalam stack yang berupa kode berikut :

```
inputStack.push(bracket);
```

Tujuannya code itu di implementasikan itu untuk memasukkan semua karakter pembuka ((, [, {) untuk masuk kedalam stack. Ini digunakan untuk simpan urutan simbolnya dimana urutan pembuka ini akan masuk ke bagian pop

```
char top = inputStack.pop();
```

Pop disini digunakan untuk cari karakter penutup dan mencari tahu karakter mana yang duluan digunakan sebagai penutup. Karena ini fungsinya memastikan sesuatu itu pasti balanced, seperti ([]) berarti apabila stack di push dan di pop, maka hasilnya pasti akan sama dan apabila tidak sama maka bisa dipastikan kalau string tidak balance

- **Algorythm Explanation**

Alur kerja algoritmanya sendiri sebenarnya cukup simpel.

1. Dibuka input scanning dan input itu akan diambil dan dimasukkan kedalam function is_balanced()
2. Input yang masuk ke dalam function akan dicek menggunakan modulus 2, tujuannya untuk langsung menentukan apakah string yang diinput invalid atau valid. Ini dikarenakan apapun yang memiliki tutup pasti genap ex : () [] {}
3. Dari situ, string di break menjadi Char dan masuk kedalam Stack baru
4. Di dalam stack ini, akan dilakukan looping pengecekan bracket, apabila ditemukan maka bracket akan di push
5. Setelah di push, maka akan dilakukan pop untuk karakter penutup untuk mengecek apakah hasil data yang di push dan yang di pop sama, apabila sama maka bisa dipastikan bahwa itu balance, dan loopingnya berlanjut
6. Apabila ditemukan adanya bracket yang tidak balance maka akan di set variable isBalanced menjadi false
7. If terakhir digunakan untuk mengecek apakah isBalanced true dan hasil stackInput sudah habis, apakah keduanya true maka pasti balance dan apabila tidak maka tidak balance

- **Correctness Argument**

Algoritmanya pasti akurat dikarenakan berbasis pada matematika dasar. Pembuka dan penutup bracket itu pasti ganjil sehingga apabila panjangnya tidak genap langsung gagal.

Selain itu, kurung yang pertama dibuka pasti yang pertama tertutup doang yang sangat sesuai dengan LIFO dimana stack ini memaksakan aturan secara langsung dan dilakukan pengecekan kedua hasil untuk memastikan hasilnya sama.

Dan terakhir, ada syarat lagi untuk memastikan hasilnya balanced dimana keduanya memiliki pembuka dan penutup. Ini tujuannya untuk menghilangkan kemungkinan adanya duplicate pembuka tanpa penutup seperti (()

- **Time & Space Complexity**

Secara complexity, ini termasuk O(n) dikarenakan dia melakukan interasi pada jumlah input yang diberikan (pengecekan string), jadinya apabila input ada 10 maka dia akan cek string sebanyak 10x

- **Test Evidence**

1. Input : ((((((((((
Expectation : Not Balanced

```
Enter the string to check if it is balanced or not:
(((((((((
The string is not balanced.
```

2. Input : {[()]} (x12)
Expectation : Balanced

```
Enter the string to check if it is balanced or not:
{{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}{[()]}
The string is balanced.
```

- **Code Reference**
  - Visual Studio Code Built In IDE Autofill (bagian for (char bracket : brackets))
  - Elvina Angelica IMT Code

II. Question Number 1I : Reverse Polish Notation (RPN) Evaluation

- **Problem Summary**

Disini problem utamanya adalah kami diminta untuk membuat fungsi evaluateRPN yang dapat menghitung hasil dari ekspresi aritmatika dalam format Reverse Polish Notation. Tantangan utamanya adalah mengolah angka dan operator secara berurutan tanpa tanda kurung

- **Stack Design**

Dalam implementasi RPN, Stack digunakan sebagai tempat penyimpanan sementara bagi angka-angka (operands). Terdapat 2 fungsi utama:

```
rpnStack.push(Integer.parseInt(token))
```

Digunakan saat program menemukan angka dalam token. Angka tersebut akan di stack dan dipaksa untuk menunggu operator yang sesuai.

```
int b = rpnStack.pop();
int a = rpnStack.pop();
```

Pop disini digunakan untuk cari karakter operator, stack mengambil dua elemen teratas.

Ini dilakukan sehingga hasil operasi apapun itu menggunakan bagi kali ataupun tambah kurang tetap bisa akurat. Kemudian hasil perhitungan akan di push kembali dalam input untuk dilanjutkan ke perhitungan selanjutnya dan ini di looping hingga angka dalam stack itu habis

- **Algorythm Explanation**

Alur kerja algoritmanya sendiri sebenarnya cukup simpel.

1. ScanningInput menggunakan Scanner dan dipecah menggunakan split untuk menghilangkan keungkinan adanya spasi dalam inputan nilai, sehingga memungkinkan untuk input hal seperti "3 10 +"
2. Input kemudian masuk ke bagian calculation dan dilakukan iterasi dan validasi ke setiap bagian stack untuk mencari mana yang berupa operator dan mana yang berupa angka
3. Setelah ditentukan apabila terdapat Operator, akan dicek apakah ada angka yang cukup dalam stack (min 2), apabila kurang maka invalid apabila cukup maka akan lanjut ke operasi
4. Disini saya menggunakan switch yang basis dari token operasi yang ada, tujuannya untuk mengurangi nested if. Selain itu apabila di deteksi ada 0 / maka akan dibuat invalid
5. Setelah perhitungan selesai dilakukan, hasilnya akan masuk kembali kedalam loop operasi hitung hingga bagian angka dan operator dalam stack habis, dalam kasus itu maka akan dilakukan print output

- **Correctness Argument**

Algoritma ini akurat karena format postfix secara alami mengikuti struktur Stack. Tidak diperlukan aturan perhitungan seperti kali lebih diprioritaskan dulu dibandingkan tambah kurang karena dalam RPN, urutan operasi ditentukan oleh posisi operator itu sendiri. Selain itu disini juga menggunakan int, sehingga tidak akan ada angka berkoma yang muncul dan ada failsafe juga dengan yield = 0. Dan di akhir ada di cek apabila stack sudah = 1 yang menyebabkan data untuk pop dan menjadi final resultnya.

- **Time & Space Complexity**

Secara complexity, ini termasuk O(n) dikarenakan dia melakukan interasi pada jumlah input yang diberikan. Setiap token diproses tepat satu kali.

- **Test Evidence**

1. Input : 2 1 + 3 *

Expectation : 9

```
Enter the expression you want :
2 1 + 3 *
Final Result : 9
```

2. Input : 10 0 /

Expectation : Division by zero is not allowed

```
Enter the expression you want :
10 0 /
Division by zero is not allowed
Final Result : 0
```

3. Input : 1000000 49 * 20 + 10 *

Expectation : 490000200

```
Enter the expression you want :
1000000 49 *    20   +   10 *
Final Result : 490000200
```

- **Code Reference**
  - Visual Studio Code Built In IDE Autofill
  - Gemini 3.1 Pro – Integer Parsing & Regex

III. Question Number III : Chrono Stack Engine (Stack Instruction Reconstruction)

- **Problem Summary**

Di sini kita berperan sebagai Time Engineer untuk menyusun ulang instruksi mesin waktu yang rusak. Mesin ini bekerja dengan prinsip Stack (LIFO) dan memiliki 3 instruksi utama: 1 (tambah unit), d (duplikasi), dan + (penggabungan). Masalah utamanya adalah instruksi + bersifat destruktif; setiap kali dilakukan penggabungan, semua elemen lain di dalam stack akan berkurang 1 unit (distorsi waktu). Tugas kita adalah mencari urutan instruksi agar menghasilkan kondisi akhir timeline yang diminta.

- **Stack Design**

Dalam kode ini, kita menggunakan dua stack utama untuk mengelola rekonstruksi:

1. targetStack digunakan untuk menyimpan angka-angka target dari input. Karena input diberikan dari dasar ke atas, kita memasukkannya ke stack agar saat di-*pop*, kita memproses elemen yang paling atas (paling terakhir di timeline) terlebih dahulu.

2. outputStack digunakan untuk menyimpan kumpulan instruksi dari setiap angka yang berhasil dikonstruksi. Karena kita membangun angka dari yang paling terakhir, kita butuh stack ini untuk membalikkan urutannya kembali agar instruksi dibaca dari awal.

- **Algorythm Explanation**

Alur kerja algoritmanya sendiri sebenarnya cukup simpel.

1. ScanningInput menggunakan Scanner dan dipecah menggunakan split untuk menghilangkan keungkinan adanya spasi dalam inputan nilai
2. Algoritma memproses dari elemen teratas karena setiap instruksi + di masa depan akan mengurangi nilai elemen di bawahnya sebesar 1, kita menambahkan nilai totalSuffixPlus ke target angka saat ini
3. Fungsi buildInstructions mengubah angka target menjadi instruksi menggunakan logika biner
4. Setiap kali satu angka selesai dibuat instruksinya, kita menghitung berapa banyak simbol + yang digunakan, lalu menambahkannya ke totalSuffixPlus untuk angka berikutnya di bawah.

- **Correctness Argument**

Algoritma ini pasti akurat karena menangani efek distorsi + secara matematis. Dengan menambahkan jumlah + yang akan datang ke angka yang sedang dibuat, kita memastikan bahwa ketika distorsi terjadi (pengurangan 1), angka tersebut akan turun tepat ke nilai target yang diinginkan.

Kemudian dengan menggunakan representasi biner menjamin bahwa angka sebesar apa pun (dalam range Long) dapat dikonstruksi dengan cepat melalui operasi duplikasi (d) dan tambah (1).

- **Time & Space Complexity**

Secara complexity, ini termasuk O(n log m) di mana N adalah jumlah elemen dalam timeline dan M adalah nilai angka terbesar. Logika biner membutuhkan waktu log M untuk setiap angka yang diinput

- **Test Evidence**

1. Input : 5 1 5 6 3 2

Expectation : 1d+1+d+d+1d+1+d+d+1d+d+d+1+1d+d+1d+

```
Enter the expression you want :
5 1 5 6 3 2
1d+1+d+d+1d+1+d+d+1d+d+d+1+1d+d+1d+
```

2. Input : 10 6 324 3 32 54 2 94 54 78 93

Expectation :
1d+d+1+d+d+1+d+1+d+1+1d+1+d+d+d+d+1+d+d+1+d+1
d+1+d+1+d+1+d+1+d+1+1d+d+1+d+d+d+1+d+1+1d+1+d
+d+d+d+d+1+1d+d+d+1+d+1+d+1d+1+d+1+d+d+1+d+1+
d+1+1d+d+d+1+d+d+d+1d+d+1+d+1+d+d+d+1d+d+1+d+
1+d+1+d+d+1+

```
Enter the expression you want :
10 6 324 3 32 54 2 94 54 78 93
1d+d+1+d+d+1+d+1+d+1+1d+1+d+d+d+d+1+d+d+1+d+1d+1+d+1+d+1+d+1+d+1+1d+d+1+d+d+d+1+d+1+1d+1+d
```

Aslinya outputnya lebih Panjang lagi namun saya cut sedikit untuk mengurangi inefisiensi pemanfaatan space

- **Code Reference**

- Visual Studio Code Built In IDE Autofill
- Gemini 3.1 Pro – Logic & Instructions

IV. Question Number IV : Cheat BsStack Based Ordering

- **Problem Summary**

Tugas utama di sini adalah merekonstruksi kode rahasia untuk membatalkan proses penonaktifan sistem Cheat Buster. Kita diberikan sekumpulan angka acak yang harus diurutkan dari yang terkecil ke yang terbesar (ascending) menggunakan mekanisme Stacking Formation Injection. Inti dari masalah ini adalah mengimplementasikan algoritma pengurutan hanya dengan menggunakan stack

- **Stack Design**

Dalam kode ini, kita menggunakan dua stack utama untuk mengelola rekonstruksi:

1. codeStack digunakan untuk menyimpan urutan angka awal yang diterima dari input. Angka angka ini akan di-pop satu per satu untuk diproses ke dalam formasi pengurutan
2. sortedStack untuk menyimpan angka-angka dalam kondisi terurut. Angka kecil akan masuk lebih dulu (berada di bawah), dan angka besar akan berada di posisi atas selama proses iterasi

- **Algorythm Explanation**

Alur kerjanya sendiri hampir sama dengan Insertion Sort dimana algoritma utamanya untuk melakukan sortingan dari kecil ke besar.

1. ScanningInput menggunakan Scanner dan dipecah menggunakan split untuk menghilangkan keungkinan adanya spasi dalam inputan nilai dan memastikan bahwa field tidak kosong
2. Input string dipecah menjadi token dan dimasukkan seluruhnya ke dalam codeStack
3. Pop codeStack dan dijadikan sebagai currentNum
4. currentNum dibandingkan dengan peek di sortedStack, apabila sortedStack lebih besar maka elemennya masuk ke codeStack kembali
5. Apabila elemennya lebih kecil, masukkan currentNum ke sortedStack
6. Proses di ulang terus menerus hingga angka

- **Correctness Argument**

AAlgoritma ini menghasilkan output yang benar karena menjamin stabilitas urutan melalui perbandingan berulang

Pada setiap langkah iterasi, sortedStack selalu dijaga dalam kondisi terurut. Setiap kali ada angka baru yang merusak urutan (lebih kecil dari puncak), elemen di atasnya disingkirkan sementara ke stack asal, sehingga posisi angka kecil tersebut dijamin berada di bawah angka yang lebih besar

Meskipun elemen dipindahkan bolak-balik antara dua stack, tidak ada data yang hilang & elemen hanya berganti posisi hingga mencapai urutan yang diinginkan

- **Time & Space Complexity**

Secara complexity, ini termasuk $O(n^2)$ karena dalam kasus terburuk dia akan sorting semua berkali kali hingga stacknya ter sortir, namun secara space dia Cuma $O(n)$ dikarenakan ruang yang dibutuhkan untuk stack sama dengan input, kita cuma membutuhkan tempat menyimpan elemennya.

- **Test Evidence**

1. Input : 42 9 17 63 28 5 74

Expectation : 5 9 17 28 42 63 74

```
Input secret code :
42 9 17 63 28 5 74
5 9 17 28 42 63 74
```

2. Input : 10 6 324 3 32 54 2 94 54 78 93

Expectation : 34 7 89 12 56 3 71 25 44 18 90 6 39 52 14

```
Input secret code :
34 7 89 12 56 3 71 25 44 18 90 6 39 52 14
3 6 7 12 14 18 25 34 39 44 52 56 71 89 90
```

- **Code Reference**
  - Visual Studio Code Built In IDE Autofill – Printing Sorted Stack

V. Question Number IV : Cheat BsStack Based Ordering

- **Problem Summary**

Tugas utama di sini adalah menghitung momentum setiap pantulan bom. Kecepatan awal dijadikan sebagai menjadi momentum awal, dan setiap pantulan berikutnya kehilangan setengah dari momentum sebelumnya hingga mencapai unit terkecil (1). Setelah dihitung maka kita harus menampilkan urutan momentum tersebut dari pantulan terkecil hingga kembali ke kecepatan awal pesawat menggunakan struktur data Stack.

- **Stack Design**

Dalam kode ini, stack utamanya digunakan untuk perhitunan matematikanya

1. Push digunakan untuk menghitung n makin mengecil ½ hingga menjadi 1. Setiao hasil pembagiannya akan masuk kedalam stack dan karena tipe stack itu LIFO, dia akan selalu terpanggilkan hingga diberikan conditioning = 1
2. Pop digunaka untuk print outputnya, dikarenakan 1 itu yang terakhir di push maka itu juga yang akan keluar menjadi output akhir, disini tidak perlu lagi pake array atau sort karena sudah efek dari LIFO sendiri

- **Algorythm Explanation**

Alur kerjanya sendiri hampir sama dengan Insertion Sort dimana algoritma utamanya untuk melakukan sortingan dari kecil ke besar.

1. ScanningInput menggunakan Scanner dan dipecah menggunakan split untuk menghilangkan keungkinan adanya spasi dalam inputan nilai dan memastikan bahwa field tidak kosong dan menjadikannya inputMomentum
2. Menggunakan while (tempNum >= 1), program terus membagi angka dengan 2, setiap hasil antara disimpan ke dalam stack
3. Setelah mencapai nilai < 1, program berhenti mengisi stack
4. Melakukan looping pada stack untuk melakukan pop hingga kosong, sehingga urutan angka yang tadinya besar-ke-kecil berubah menjadi kecil-ke-besar saat dicetak.

- **Correctness Argument**

Algoritma ini akurat karena memanfaatkan sifat dasar Stack untuk pembalikan urutan. Pola pantulan yang membagi dua secara konsisten direpresentasikan dengan tempNum / 2 yang kemudian kita akan mendapatkan stack dengan ukuran dari rendah ke tinggi dan conditioning =1 dipastikan bahwa pantulan terakhir dicatat disana

- **Time & Space Complexity**

Secara complexity, ini termasuk O(log n) karena input selalu dibagi dua di setiap langkah, jumlah operasi yang dilakukan adalah logaritmik terhadap nilai n.

- **Test Evidence**

1. Input : 500

Expectation : 1 3 7 15 31 62 125 250 500

```
Input Velocity (n): 500
1 3 7 15 31 62 125 250 500
```

3. Input : 1000000

Expectation : 1 3 7 15 30 61 122 244 488 976 1953 3906 7812 15625 31250 62500 125000 250000 500000 1000000

```
Input Velocity (n): 1000000
1 3 7 15 30 61 122 244 488 976 1953 3906_7812 15625 31250 62500 125000 250000 500000 1000000
```

- **Code Reference**